

Competitive Programming

Leonardo Scoppitto
September 2025

Contents

1	21/09/2025	1
	1.1 Sliding window maximum	1
2	27/10/2025	2
	2.0.1 Triplets	2
3	Binary search for a problem	5
4	Social Distancing	6
5	100 prisoners problem	7
6	Duplicate elements in array	8
	6.1 Floyd's cycle finding	9
7	Majority element	11
	7.1 Boyer-Moore algorithm	11
8	Dynamic programming	12
	8.1 Robberies – From Facebook	12
	8.2 Snakes and Ladders	12
	8.3 Maximum Cost Path	13
	8.4 Longest Common Subsequence	14
	8.5 0/1 Knapsack problem	15
9	Longest Increasing Subsequence (LIS)	16
10	Greedy algorithms	18
	10.1 Activity selection problem	18
	10.2 Job sequencing	19
	10.3 Magic number	19
	10.4 Lexicographically Maximum Subsequence	20
	10.5 Woodcutter	20
	10.6 Queue of people	21
	10.7 Boxes	22

1 21/09/2025

1.1 Sliding window maximum

You are given an array of integers `nums`, there is a sliding window of size `k` which is moving from the very left of the array to the very right. You can only see the `k` numbers in the window. Each time the sliding window moves right by one position.

Example 1:

Input: `nums = [1,3,-1,-3,5,3,6,7]`, `k = 3` Output: `[3,3,5,5,6,7]` Explanation:

Window position	Max
-----	-----
<code>[1 3 -1] -3 5 3 6 7</code>	<code>3</code>
<code>1 [3 -1 -3] 5 3 6 7</code>	<code>3</code>
<code>1 3 [-1 -3 5] 3 6 7</code>	<code>5</code>
<code>1 3 -1 [-3 5 3] 6 7</code>	<code>5</code>
<code>1 3 -1 -3 [5 3 6] 7</code>	<code>6</code>
<code>1 3 -1 -3 5 [3 6 7]</code>	<code>7</code>

Example 2:

Input: `nums = [1]`, `k = 1` Output: `[1]`

2 27/10/2025

Given an array of non-negative integers, support:

- $\text{Add}(i, v)$
- $\text{RangeSum}(i, j)$
- $\text{Search}(s) \rightarrow$ reports the position of the shortest prefix of A s.t. $\sum_{k=1}^i A[k] \geq s$

To support the $\text{Search}(s)$ we can naively try to binary search the solution comparing each time the sum from the start to the index, store it and then continue left or right. This takes $\Theta(\log^2 n)$

Using a Segment tree, we can start from the top and check the root of the left sub-tree:

- If the root is $> s$ we can continue searching the solution there
- Else, we skip entirely the left sub-tree and start searching the solution in the right sub-tree, updating the target to $s - R_l$, where R_l is the left sub-tree root value.

Given an array of integers support:

- $\text{Add}(i, v)$
- $\text{\#occ}(i, j, x) \rightarrow$ the number of occurrences of x in the range $[i, j]$

Of course HashMap, but I did not understand how to combine it with the Segment Tree. Maybe in each node we store an HashMap with all the values of their grand-...-grand children (up to the leaves).

So the run time is $\Theta(\log n)$, while the space is $\mathcal{O}(n \times \log n)$

We can use this trick also to compute the $\text{successor}(i, j, x)$ (finding the index k of the first element s.t. $k \in [i, j] \wedge A[k] = x + 1$) embedding a BST inside each node:

- Compute the successor in every total overlapping node
- Combine taking the minimum

Run time is $\Theta(\log^2 n)$, while the space is $\Theta(n \times \log n)$.

We can improve the query time up to $\Theta(\log n)$ with the fractional cascading technique: we will see later.

2.0.1 Triplets

Given $A[0, n-1]$ with integers count the number of triplets $i, j, k, 0 \leq i < j < k < n$ such that $A[i] < A[j] < A[k]$

Example

$$A = [1, 2, 3, 4, 1] \rightarrow 4$$

$$[[1, 2, 3], [1, 2, 4], [2, 3, 4], [1, 3, 4]]$$

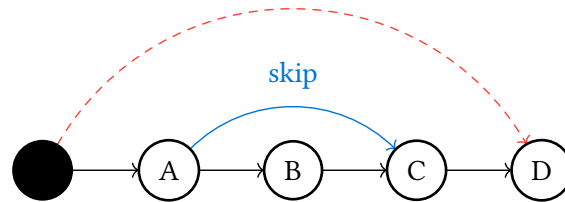
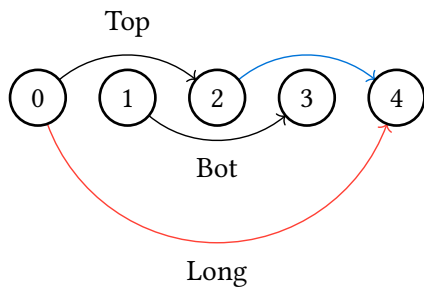
Solution

The naive solution considers all the possible triplets: $\Theta(n^3)$ A smarter solution that runs in $\Theta(n^2)$ is considering a central element (each one in the list) and count the elements lower and greater, respectively in the left and right subarray, then consider the product:

$$\left[\underbrace{1, 2}_{k=2}, \underbrace{3}_{k'=1}, \underbrace{4, 1}_{k'=1} \right] \rightarrow k \times k' = 2 \rightarrow \text{this is the number of element having 3 as the middle element.}$$

Repeat for every element

There is an even smarter solution in $\Theta(n \log n)$, ask Luca.

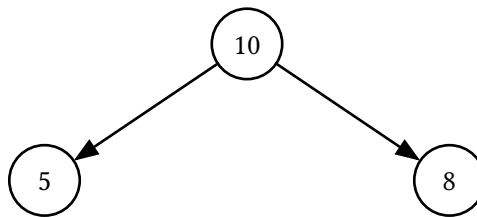


DP Array with Highlights

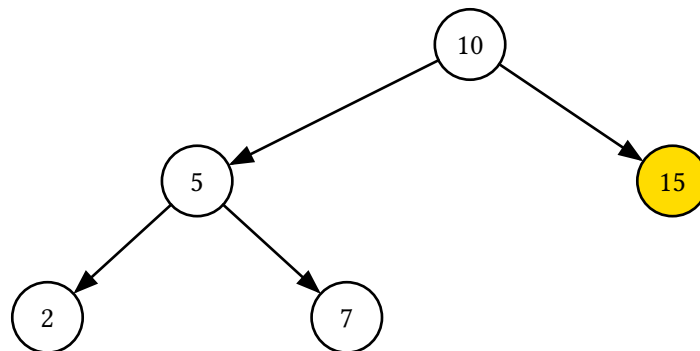
10	20	30	40	inf
0	1	2	3	4

1	2	3
---	---	---

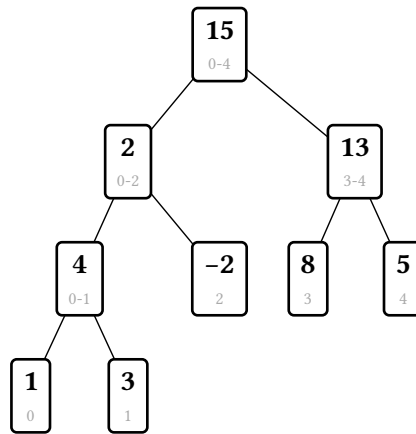
Simple Tree



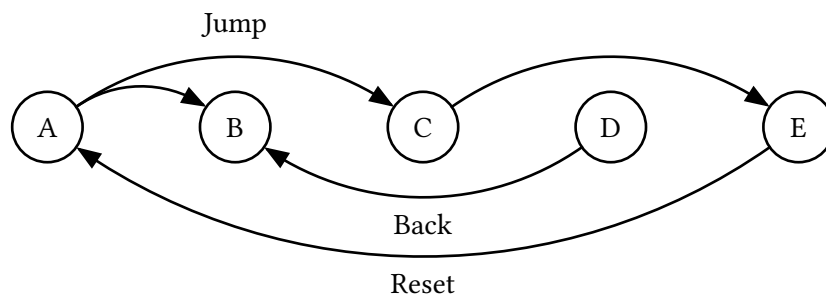
Complex Tree



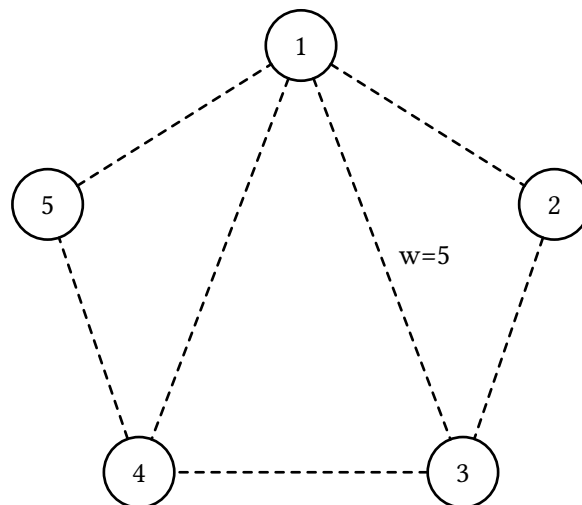
Sum Segment Tree



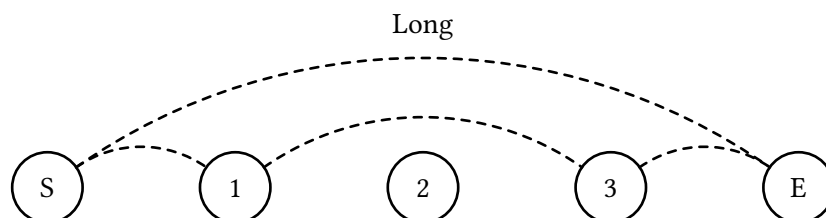
Linear Graph (Auto-Bending)



Circular Graph



Rainbow Arch Graph



3 Binary search for a problem

I can use BinarySearch to find the solution of a problem which can be modeled using a monotone predicate over a range. For example, the predicate $x \leq 3$ produces the series:

T	T	T	T	F	F	F	F
0	1	2	3	4	5	6	7

An application of the binary search for a problem can be the compute of $\text{sqrt}(v)$ ($\lfloor \sqrt{v} \rfloor$):

- Define the candidates $\rightarrow x \in [0, v]$
- Search the correct candidate using a predicate $\rightarrow x^2 \leq v$
- `BSR(0, v, lambda x, v: x**2 <= v)`

Here's an implementation:

```
#include <stdbool.h>
#include <stdint.h>
#include <stdio.h>
#define SQRTOF 1209378128ULL

typedef bool (*Predicate)(int64_t);

bool is_sqrt(int64_t x) { return x * x <= SQRTOF; }

bool binary_search_range(int64_t low, int64_t high, Predicate pred,
                        int64_t *result) {
    bool found = false;

    while (low < high) {
        int64_t middle = low + (high - low) / 2;

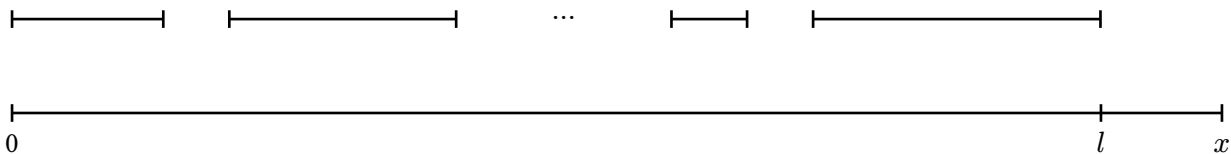
        if (pred(middle)) {
            // Predicate is true: this is a candidate, but we check higher values
            low = middle + 1;
            *result = middle;
            found = true;
        } else {
            // Predicate is false: the answer must be lower
            high = middle;
        }
    }

    return found;
}

int main() {
    int64_t answer = -1;

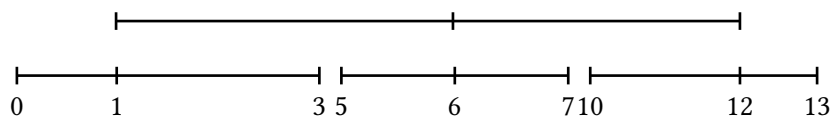
    // Search in range [0, SQRTOF)
    binary_search_range(0, SQRTOF, is_sqrt, &answer);
    printf("FLOOR(SQRT(%lld)): %ld\n", SQRTOF, answer);
    printf("%ld*%ld: %ld\n", answer, answer, answer * answer);
    return 0;
}
```

4 Social Distancing



Given a set of n intervals, find c points in the intervals such that the smallest distance between consecutive points is maximized.

Example:



As we can see, with $c = 3$, choosing the points $(1, 6, 12)$ gives us two segments long 5 and 6 units respectively, leaving us with the smallest distance $d = 5$.

To solve this problem we can use the binary search for an answer technique:

- The range is defined by be the smallest maximum distance achievable $d \in [0, l]$
- The predicate $\text{Pred}(d')$ is true iff it is possible to find c points at distance at least d' , which is monotone

RIGUARDA

5 100 prisoners problem

100 prisoners are offered freedom if they win at this game:

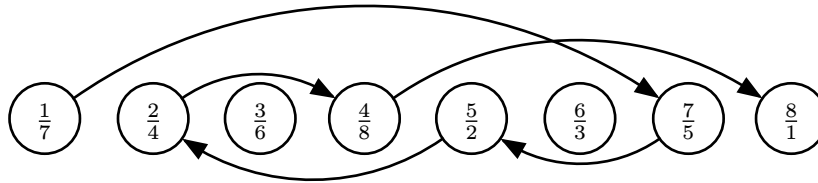
- Every prisoner is assigned a number 1..100.
- There are 100 drawers, each containing a number in the range 1..100.
- Iff every prisoner manage to find its number in a drawer in up to 50 attempts, they are all set free.
- Prisoners cannot communicate during the game, but they are allowed to set a strategy.

If they agree to open 50 drawers uniformly at random, the probability of winning is $100 \times \mathbb{P}(\text{a prisoner wins}) = \left(\frac{1}{2}\right)^{100} = 0.\underbrace{00\dots008}_{33 \text{ zeros}}$, so the chances are rather low.

To improve the probability we can use an adaptive strategy based on the content of each drawer:

- Every prisoner picks the drawer numbered with its own number.
- If the prisoner picks the right number, 😊, otherwise he opens the drawer with the number he found.

Note that with this strategy, if they could open all the drawer they would eventually win following just the numbers, since the only possible cycle would be the one containing a permutation of some of the numbers (no duplicates). Consider prisoner #1 ($\frac{\text{drawer}}{\text{content}}$):



Let's see the chances of the prisoners this time:

- A prisoner wins if its cycle is ≤ 50 long.
- $\mathbb{P}(\text{all win}) = \mathbb{P}(\text{A random permutation has no cycle longer than 50})$.
- We need to count the number of permutation with a cycle of length > 50 .
- Cycles with the same elements, but different starting points are considered equal and counted only once.

$$\binom{100}{l} \times \underbrace{(l-1)!}_{-1 \rightarrow \text{No order}} \times (100-l)!, \quad l > 50$$

$$\sum_{l=51}^{100} \frac{100!}{l} = 100! \times \sum_{l=51}^{100} \frac{1}{l} = 100! \times \left(\frac{1}{51} + \frac{1}{52} + \dots + \frac{1}{100} \right) =$$

$$100! \times (\mathcal{H}_{100} - \mathcal{H}_{50}), \quad \text{where } \mathcal{H}_i \text{ is the } i\text{-th harmonic number}$$

$$\mathbb{P}(\text{win}) = 1 - (\mathcal{H}_{100} - \mathcal{H}_{50}) \approx 0.3118\dots$$

6 Duplicate elements in array

$A[0, n]$ is an array of $n + 1$ elements in $\{0, \dots, n - 1\}$. By the pigeonhole principle¹, there must be at least one duplicate, so the task is to find one duplicate element.

1	3	5	6	5	2	0	4	7
0	1	2	3	4	5	6	7	8

Our main goal is to use as little space as possible as we need to scan every element anyway (worst case).

Solution #1 $\mathcal{O}(n)$ time, $\mathcal{O}(n)$ space:

We scan the array and insert every element in an hashset and when we find a duplicate we stop and report it.

Solution #2 $\mathcal{O}(n)$ time, $\mathcal{O}(n)$ space:

Similarly to the solution #1, we can use a direct access table, creating a bitarray and setting $b_i = 1$ when we find the element i , then report the duplicate.

Solution #2 $\mathcal{O}(n \log n)$ time, $\mathcal{O}(1)$ space:

We can do better reconstructing every element of the array bit by bit in $\Theta(\log n)$ passes:

1. We scan and keep track of the number of 0s and 1s in b_0 , in the example $(4, 5) \rightarrow$ we have an unbalanced number of 1s, so, we set the first bit of the duplicate element to 1.
2. We scan the second bit and still keep track of zeroes and ones, but discarding the elements with $b_0 = 0$. In our example we have 3 zeroes and 2 ones, so we set the second bit of the duplicate element to 0.
3. We now know the drill: we consider elements with prefix $b_0 b_1 = 10$ and in the example we set $b_2 = 1$

A	1	3	5	6	5	2	0	4	7
b_0	0	0	1	1	1	0	0	1	1
b_1	0	1	0	1	0	1	0	0	1
b_2	1	1	1	0	1	0	0	0	1

Duplicate element

b_0	b_1	b_2
1	0	1

Example 1: Solution 2 example.

Solution #3 $\mathcal{O}(n)$ time, $\mathcal{O}(1)$ space:

If we don't mind destroying the original array, we can use an approach similar to the one of the 100 prisoners problem:

1. We start from index n since $n \notin [0, \dots, n - 1]$ and we read $A[n]$.
2. We access $A[A[n]]$ and we compare it to $A[n]$ if they are equal, we are happy, otherwise we set $A[A[n]] = A[n]$ access $A[A[A[n]]]$.
3. Repeat 2 until the duplicate is not found.

¹In mathematics, the pigeonhole principle states that if n items are put into m containers, with $n > m$, then at least one container must contain more than one item.

Here's a python snippet with the code to implement this:

```
A = [1,3,5,6,5,2,0,4,7]
```

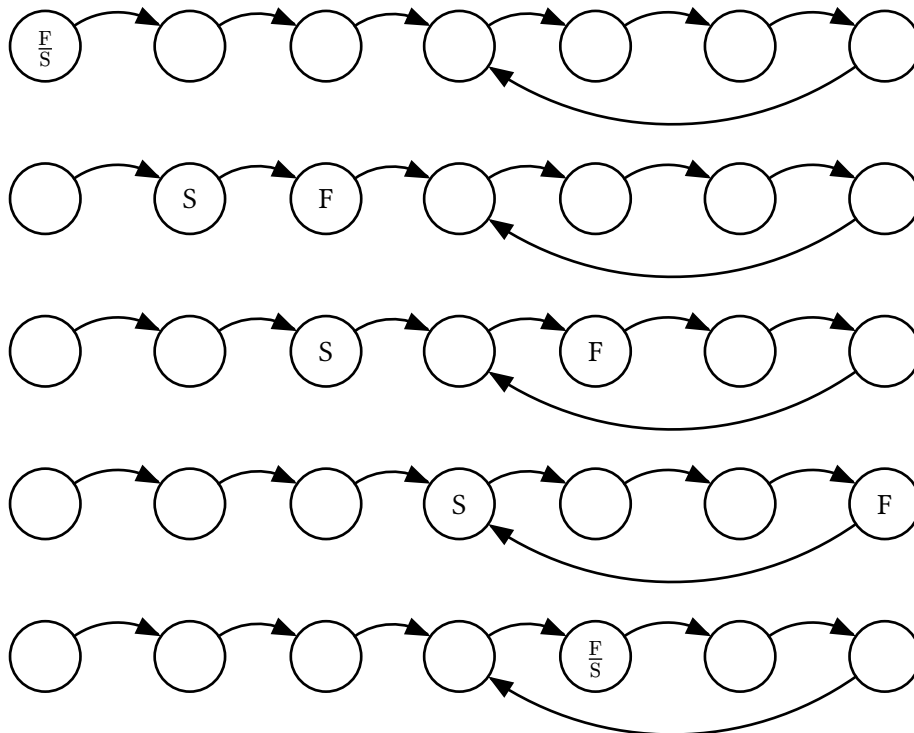
```
def find_dup(A: list[int], idx: int) -> int:
    e = A[idx]
    if A[e] == A[idx]:
        return A[e]
    next_idx, A[e] = A[e], A[idx]
    return find_dup(A, next_idx)

if __name__ == "__main__":
    print("Duplicate element is", find_dup(A, len(A)-1))
```

6.1 Floyd's cycle finding

The last solution can be generalized and applied to linked data structures to find whether there is a loop: we can scan the list using two pointers, one **fast** (moves 2 steps at a time) and one **slow** (moves just one position). Eventually, the two pointers will meet and there you will know that a loop does exist. This technique costs $\mathcal{O}(1)$ space and, unlike the last solution, it does not destroy the list.

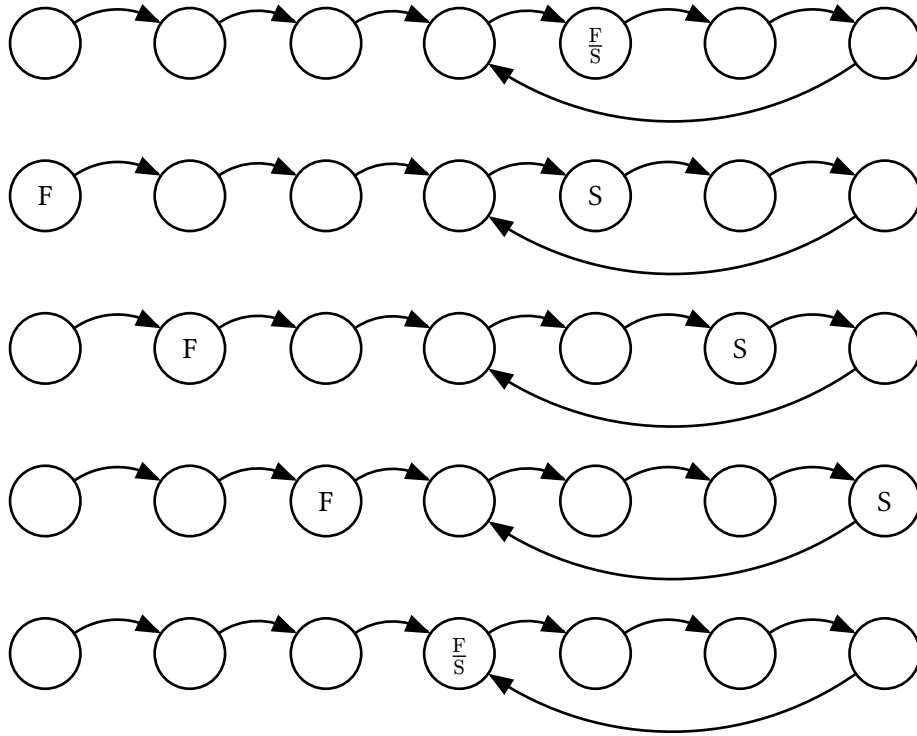
Let's see this step by step:



Now we know that a loop exists, because the two pointers met at the fifth element, but how can we get the position of the beginning of the cycle? We need to add some more rules to the game:

- The **fast pointer** has to move **first**, otherwise there are some cases where they will miss each other.
- When they meet, reset **F** to the first position and **set the speed to 1**
- Now make them move again and they will magically meet at the beginning of the cycle

Let's see this in action starting from where we left:



Here they are at the beginning of the loop! Bringing this back to the previous problem, now we know that **there is a duplicate** and **where** it is. This is not black magic, here's why it works:

- When they met the first time, **F** moved $2m$ position, while **S** just m .

- $$m = \underbrace{a}_{\text{outside the cycle}} + \underbrace{\hat{b}}_{\text{inside the cycle}} \rightarrow 2m = a + b + k \times \underbrace{l}_{\text{loop length}}$$

- $$2a + 2b = a + b + k \times l \rightarrow \underbrace{a}_{\text{F steps}} = \underbrace{k \times l - b}_{\text{S steps}}$$

- Now, resetting the fast pointer to 0 and making them move at the same speed, they will have to cover the same distance before reaching the start of the cycle, where they will meet

RICONTROLLA I CONTI

7 Majority element

Given the array $A[1\dots n]$, find, if any, the element that occurs at least $\frac{n}{2} + 1$ times.

Solution #1 $\mathcal{O}(n \log n)$ time, $\Theta(n)$ space:

Sort the array and count.

Solution #2 $\mathcal{O}(n)$ time, $\Theta(n)$ space:

Use an hashmap to store, for each element, the frequency and then take the max.

7.1 Boyer-Moore algorithm

This problem can be solved in $\Theta(n)$ time and $\Theta(1)$ space, using a partial counter.

Note: the algorithm works iff **there is** a majority element!

Let's see an example:

A	5	3	1	1	2	3	1	1	1
---	---	---	---	---	---	---	---	---	---

counter 1 1 1 2 1 1 1 2 3

- We keep a candidate c and a counter.
- We process the array left to right using the following algorithm.

```
def find_majority_elem(A: list[int]) -> int:
    for i in len(A):
        if A[i] == c:
            counter += 1
        else:
            counter -= 1
        if counter == 0:
            c = A[i]
            counter = 1
    return c
```

Code 1: Boyer-Moore algorithm.

Why it works? If there is a majority element, we can use their instance to *kill* other elements:

Index	Value	Action	Candidate	Count	Logic
0	5	New Cand	5	1	Set 5 as leader.
1	3	Mismatch	5	0	3 “kills” the 5.
2	1	New Cand	1	1	Count is 0. 1 takes the lead.
3	1	Match	1	2	1 reinforces itself.
4	2	Mismatch	1	1	2 “kills” a 1.
5	3	Mismatch	1	0	3 “kills” the last 1.
6	1	New Cand	1	1	Count is 0. 1 takes lead again.
7	1	Match	1	2	1 reinforces itself.
8	1	Match	1	3	1 reinforces itself.

8 Dynamic programming

Just to refresh the memory from operational research (?), we can model optimization problems on a graph by assigning a cost (min optimization problem) or a benefit (max opt. prob.) to the edges of a graph and then compute the largest/shortest path using the Dijkstra algorithm. Also, if we manage to model the problem into a DAG³ the algorithm is even more efficient.

Processing the edges from left to right: if $(i, j) \in E, c_j = \min / \max(c_i, c_i + w_{ij})$

The **time complexity** is $\mathcal{O}(|V| + |E|)$ time

8.1 Robberies – From Facebook

You are a robber and you have to maximize the value of your heist:

- There are n houses with an associated r_n value
- If you steal from house i , you cannot enter houses $i - 1$ and $i + 1$

We can model this problem into a DAG and then compute the longest path aimed to maximize the benefit (the return from each robbery):

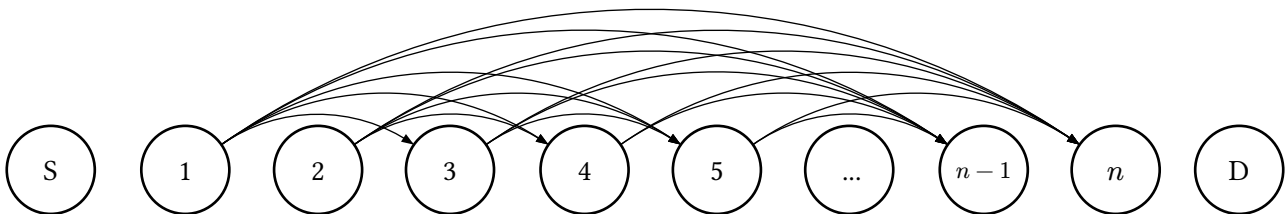


Figure 1: From the Source we can go to every node and, from every node, we can reach the Destination.

We can then assign the return value r_i to each edge and in each node we compute the total revenue up to that node for every possible path in $\Theta(n^2)$.

Actually, we can improve this solution by making a smarter choice and discard some edges: we can choose either to go to $i + 2$ or $i + 3$ depending on which one gives us the most value, but it would be a waste to go past $i + 4$ because we would throw away $i + 2$ even if it's *legal* to take it.

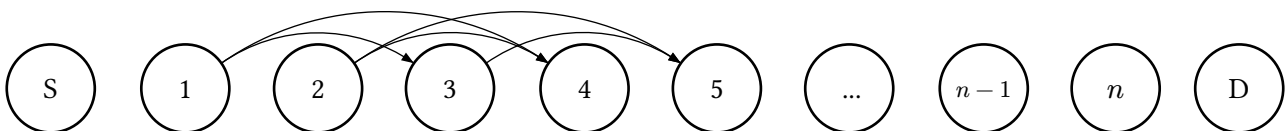


Figure 2: Improved version of the DAG.

Now the number of paths to compute is significantly lower and this solution takes $\Theta(n)$ **time**.

8.2 Snakes and Ladders

Snake and Ladders is (or maybe was) a popular board game where the pawn has to reach the last cell of a 100×100 matrix avoiding snake, which make it *ssslip* back, and trying to catch ladders, which make it *JUMP UP* some positions.

Compute the series of throws that make you win in the fewest moves.

²Ricerca Operativa.

³Direct Acyclic Graph.

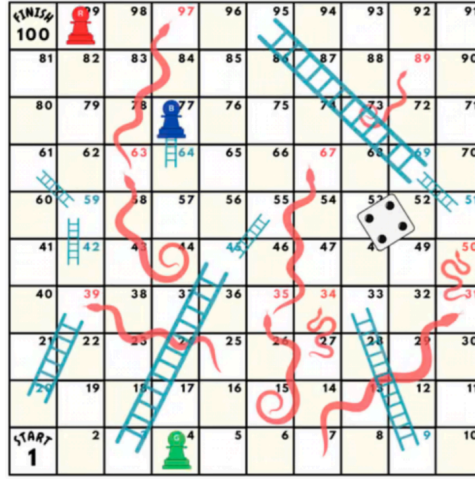


Figure 3: Snakes and Ladders board.

Also this problem can be modeled as a DAG where the node i represents the cell i :

- Each node i has 6 edges connecting it to the $i + [1..6]$ node for obvious reasons.
- If in a cell there is a snake or a ladder, the node is connected to the respective node cell forward or backward accordingly to the item

Note: we should artificially prune the graph to remove any cycle.

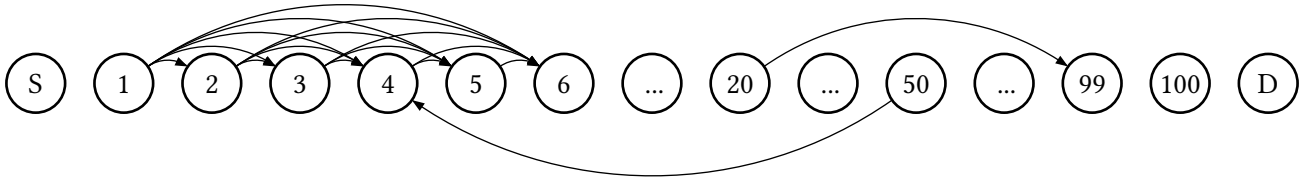


Figure 4: An random example of the how the graph looks like.

8.3 Maximum Cost Path

Given A a matrix of integers $n \times m$, compute the max path of minimal cost from the top left to the bottom right cells using only the $\downarrow$ and $\rightarrow$ moves. With dynamic programming this is actually easy because you just report the allowed moves and find the right path using the source value as a cost of the edge:

$$\begin{pmatrix} 1 & 2 & \dots & 6 \\ 0 & 0 & \dots & 2 \\ \vdots & \vdots & \ddots & \vdots \\ 1 & 7 & \dots & 7 \end{pmatrix} \rightarrow \begin{pmatrix} 1 & \xrightarrow{\underline{1}} & 2 & \xrightarrow{\underline{2}} & \dots & 6 \\ \downarrow \underline{1} & & \downarrow \underline{2} & & \dots & \downarrow \underline{6} \\ 0 & \xrightarrow{\underline{0}} & 0 & \xrightarrow{\underline{0}} & \dots & 2 \\ \downarrow \underline{0} & & \downarrow \underline{0} & & \dots & \downarrow \underline{2} \\ 1 & \xrightarrow{\underline{1}} & 7 & \xrightarrow{\underline{7}} & \dots & 7 \end{pmatrix}$$

Figure 5: Transform the matrix in a DAG.

The **time complexity** is $\Theta(n \times m)$.

8.4 Longest Common Subsequence

Given S_1 ($|S_1| = n$) and S_2 ($|S_2| = m$), find the length of the longest common subsequence $\text{LCS}(S_1, S_2)$.

S_1	A	B	C	B	D	A	B
S_2	B	D	C	A	B	A	

Example 2: $|S_1| = 7$, $|S_2| = 6$, $\text{LCS}(S_1, S_2) = AB$

Solution #1 $\Theta(n \times m)$ time:

We divide the problem in multiple **subproblems**: instead of computing the LCS for the entire sequences we start to compute it for prefixes of S_1 and S_2 . Let's call $\text{LCS}(i, j)$ the length of the LCS between $S_1[1...i]$ and $S_2[1...j]$. To **combine** subproblems we have to check the last character of the sub-strings and distinguish two cases:

1. **if** $c_1 == c_2$:
 $\text{LCS}(i, j) = \text{LCS}(i-1, j-1) + 1$
2. **if** $c_1 != c_2$:
 $\text{LCS}(i, j) = \max(\text{LCS}(i, j-1), \text{LCS}(i-1, j))$

Finally the **recurrence** step is:

```
def LCS(i, j):
    if i == 0 or j == 0:
        return 0
    if S[i] == S[j]:
        return LCS(i-1, j-1) + 1

    return max(LCS(i, j-1), LCS(i-1, j))
```

This is also referred as a **top-down** approach.

Solution #2 (also **bottom-up** approach):

This approach involves building a table row-wise computing the value using the row above:

match +1 ↘	if max ↓
if max →	Next item

j \ i	∅	B	D	C	A	B	A
∅	0	0	0	0	0	0	0
A	0	0	0	0	0+1	1	1
B	0	1	1	1	1	2	2
C	0	1	1	2	2	2	2
B	0	1	1	2	2	3	...
D	...	...	...	...	...	...	...
A	...	...	...	...	...	...	...
D	...	...	...	...	...	...	...

Example 3: Table built using the bottom up approach: as we can see in case of a **match** we take the elements in the row above at position $(i-1, j-1)$. Instead when we have a **mismatch**, we inspect the cells $(i-1, j)$ and $(i, j-1)$ and we take the **max**, **discarding** the lower.

8.5 0/1 Knapsack problem

You are given n items and a knapsack with capacity c . Each item i has a value v_i and a weight w_i .

The **goal** is to select a subset of items that maximizes the total value respecting the weight constraint c .

This is a well known **NP-hard** problem, but it can be reduced to polynomial time under specific circumstances.

Weight	Value
1	1
3	4
4	5
5	6

Example 4: We can select the first and last element and get a value of 8 or the two middle elements and get a value of 9.

9 Longest Increasing Subsequence (LIS)

Given a sequence S of n elements, find the LIS of s

10	22	9	21	33	50	41	60	80
0	1	2	3	4	5	6	7	8

An example of increasing subsequence is:

10	22	9	21	33	50	41	60	80
----	----	---	----	----	----	----	----	----

While one of the possible LIS is:

10	22	9	21	33	50	40	60	80
----	----	---	----	----	----	----	----	----

Solution #1 ($\Theta(n^2)$ time):

We can consider as the subproblem the LIS of a portion of the array and then extend it. This approach however does not work because we do not know which is the last element of a LIS. Imagine the following sequence and where the prefix is up to the index 3:

1	2	10	100	0	0	0	0	...
0	1	2	3	4	5	6	7	8

RIGUARDA

The actual solution is to define the LIS as follows:

$LIS(i)$ = is the LIS of $S[0...i]$ which ends with $S[i]$

And then set the recurrence to:

$LIS(i) = 1 + \max\{LIS(j) | 1 \leq j \leq i - 1 \wedge S[j] < S[i]\}$

$\frac{10}{1}$	$\frac{22}{2}$	9	21	33	50	41	60	80
0	1	2	3	4	5	6	7	8

Solution #2 ($\theta(n^2)$ time):

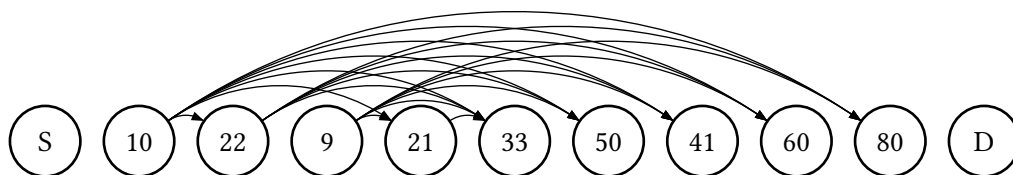


Figure 6: Not all the edges are present

Solution #3 ($\theta(n \log n)$ time):

We say that a position i **dominates** a position j if $S[i] < S[j] \wedge LIS(i) \geq LIS(j)$.

S	10	22	9	21	33	50	4	60	80
LIS	1	2	1	2	3	4	5	5	6

Example 5: **9** dominates **10** as they have the same LIS, but $9 < 10$.

Note that:

- Every values of the LIS are represented from 1... LIS
- $\exists!$ a dominating position for every value of the LIS

We can now build a table D of dominating positions where, for every LIS, we store the dominating value and then we binary search for the predecessor.

LIS	1	2	3	4	5	6
Val	10	22				
Val	10	22				
Val	9	22				
Val	9	21	34	50		
Val	9	21	33	50		
Val	9	21	33	41	60	80

Example 6: Every update of the dominating table.

Let's see this step by step:

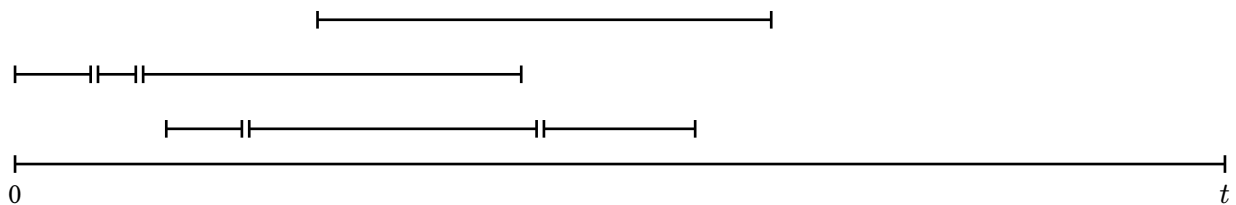
1. We insert 10, since it is the first element $\rightarrow \text{LIS} = 1$
2. BinarySearchPred(22) $\rightarrow$ we find 10 at $\text{LIS} = 1$ so we insert 22 at $\text{LIS} + 1 = 2$
3. BinarySearchPred(9) $\rightarrow$ We do not find anyone, so we check item at $\text{LIS} = 1 \rightarrow$ since $9 < 10$ we replace it.
4. BinarySearchPred(21) $\rightarrow$ We find 9 at $\text{LIS} = 1$, so we check $\text{LIS} + 1 = 2$. Replace 22 $\rightarrow$ 21
5. Insert 33 at $\text{LIS}(2) + 1 = 3$
6. Insert 50 at $\text{LIS}(3) + 1 = 4$
7. BinarySearchPred(41) $\rightarrow$ Replace 41 $\rightarrow$ 50
8. Insert 60 at $\text{LIS}(4) + 1 = 5$
9. Insert 80 at $\text{LIS}(5) + 1 = 6$

To perform every insert we binary search for the predecessor in the value row and we replace the entry at $\text{LIS} + 1$ if the current item is lower than the already inserted entry.. This takes $\mathcal{O}(n \times \log n)$ time.

10 Greedy algorithms

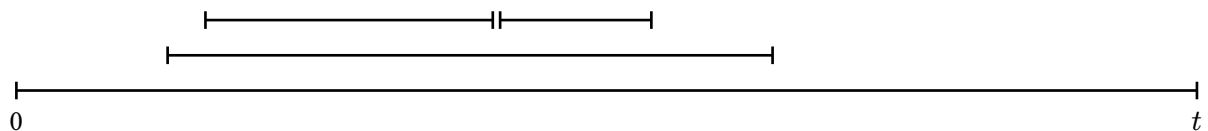
10.1 Activity selection problem

Given n activities you have to select the largest number of non-overlapping activities.



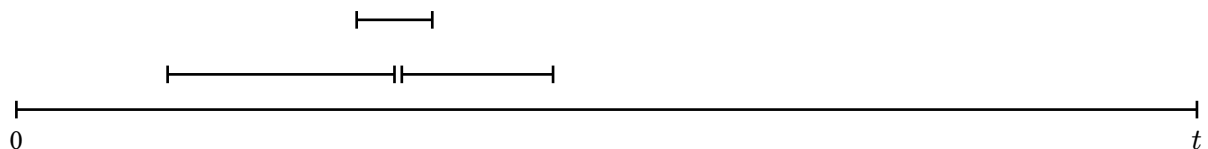
Solution #1:

We can select the activities by the smallest starting time (sorting by start time) and then select the next one that does not overlap. This is not optimal as we can see in the graph below, because we would select just one activity:



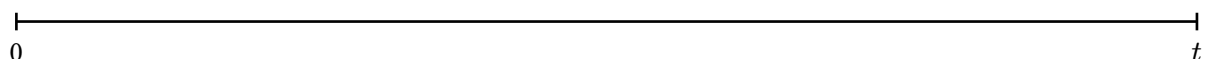
Solution #2:

Select smallest first. Also this does not work because in a situation like this, you would select only one activity.



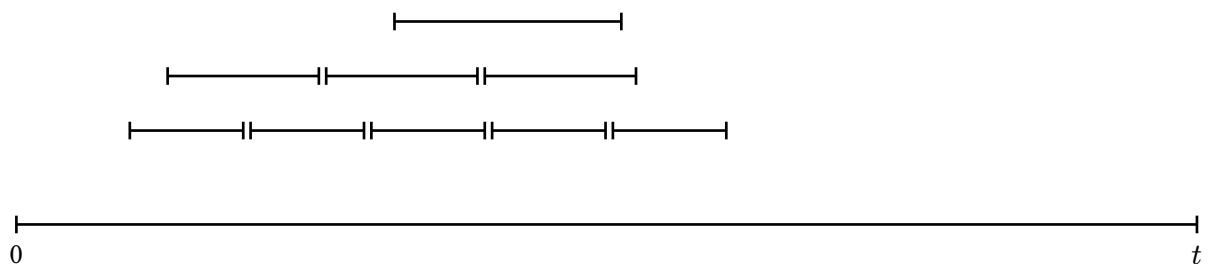
Solution #3:

Select by fewer overlaps. It is still not optimal. Find a counter example.



Solution #4:

Sort by ending time and finish as early as possible to start a new activity.

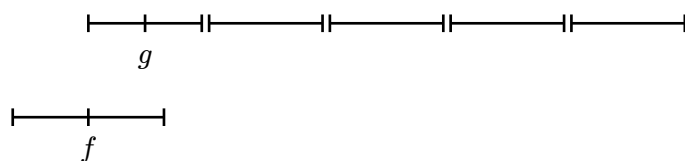


We sort by ending time in $\mathcal{O}(n \log n)$ and then scan the activities in $\Theta(n)$ and select the one that both end first and does not overlap with the last selected activity.

Lemma: there exists at least one optimal solution which includes the first ending activity f .

Proof:

Given an optimal solution, consider this



We use the *exchange* argument, which states that if you...

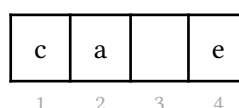
10.2 Job sequencing

We are given jobs identified by $\langle \text{JobId}, \text{Deadline}, \text{Profit} \rangle$. Every job takes 1 unit of time. You can execute a job if you can execute it before deadline and then you can take the profit. You are given a timetable and you have to fit the most rewarding job there.

JobId	Deadline	Profit
a	2	100
b	1	19
c	2	27
d	1	25
e	4	15

We can sort by deadline in decreasing order and then select the most profitable: if we can choose, we place it in the rightmost position to leave space for other jobs before it.

JobId	Deadline	Profit
a	2	100
c	2	27
d	1	25
b	1	19
e	4	15



Profit: 142

The slots can be represented in a BST and we can search for the predecessor (deadline). If there are many empty slots, use a BST on intervals.

10.3 Magic number

A magic number is formed by the concatenation of numbers, 1, 14, 144. For example 14141 is a magic number, while 1444 is not.

Determine if a number is a magic one in $\mathcal{O}(d)$, where d is the number of digits.

Solution #1:

This is my solution but it is not greedy:

```
def is_magic(S: str) -> bool:
    counte = 0
```

```

for c in S:
    if c not in ('1','4'):
        return False
    if c == '1':
        counter = 1
    if counter == 0:
        return False
    elif c == '4':
        counter += 1
    if counter > 3:
        return False
return True

```

Lempel-Zif parsing (1977) RIGUARDA NON HO CAPITO

Let's take the string ababbab, we can initialize new strings and then store the reference to that string: (0,a)(0,b)(2,1)(3,2), where (0, char) is a new item, and (n,m) is the (length, reference from start).

10.4 Lexicographically Maximum Subsequence

Consider a sequence such as ababba, the lexicographically biggest string for example babb < bbba. So, first we have to maximize the length and then maximize the lexicographical order. If I have aabaa I select baa, while if I have aabaab I select bb.

Solution #1:

I start from the end of the string and take the last symbol then I scan the string and:

- If the next symbol is smaller I discard it
- If its greater or equal the last symbol I take it

Take abbcbbccacbbcbbaaba and start from the end:

- a
- ba
- bba
- cbba
- ccbba
- ccbba
- cccbba

The algorithm can be used to find the Pareto Optimal selection see notes We can sort by x the points and then take the largest y seen so far, like the character.

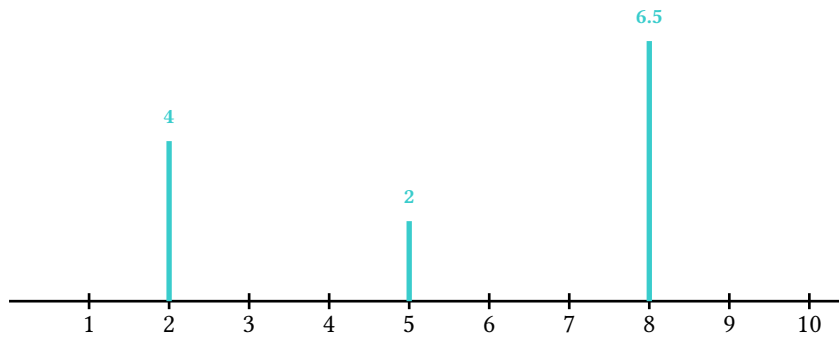
10.5 Woodcutter

There are n trees, each one with a height h and plated at position p .

When we cut a tree we can decide either to let it fall left or right, but it cannot fall on another tree, even if it has already been cut, also they cannot overlap also when they lies down.

The goal is to cut as much trees as possible.

We start from the beginning and we let the tree fall on the left, then if we can, we let the tree fall on the left, otherwise on the right. If we can, we cut.



10.6 Queue of people

For each person i we know:

- The height h_i .
- The number of people in front a_i that are taller ($\geq$).

Based on that information you have to reconstruct the queue.

Example:

a_i	0	2	0	0
h_i	a	b	c	d

→

a_i	0	0	0	2
h_i	a	c	d	b

a_i	7	4	7	5	6	5
h_i	0	4	1	0	1	2

→

a_i	7	4	7	5	6	5
h_i	0	4	1	0	1	2

We can sort by height in decreasing order, for the same height by value of a :

a_i	7	4	7	5	6	5
h_i	0	4	1	0	1	2

→

a_i	7	7	6	5	5	4
h_i	0	1	1	0	2	4

Then we start from the left and we place every person in the right spot:

a_i	7	7	6	5	5	4
h_i	0	1	1	0	2	4

→

a_i	7	7
h_i	0	1

→

a_i	5	7	6	7
h_i	1	0	1	1

FINISCI PROCEDURA

This takes $\mathcal{O}(n^2)$:

```
answer = []
for h, a in people:
    # The insert takes O(n) under the hood
    answer.insert((h,a), a)
```

A more efficient approach would be use a linked list⁴ where we can insert an element in the middle in $O(1)$, but how can we track the references to place it?

RIGUARDA REGISTRAZIONE A QUESTO PUNTO

The answer is: SkipList. Check AE notes.

10.7 Boxes

We have n boxes and for each box we have:

- The weight w_i
- The durability d_i

You can put up to weight d_i on top of box i .

The goal is to build the highest tower possible with (some) of the boxes.

Given the existence of an optimal ordering, I define $d_p[i][k]$ the minimum weight of a tower of height k with the first i boxes (this is the dynamic programming part).

$$d_p[i][k] = \min \begin{cases} d_p[i-1][k] \\ d_p[i-1][k-1] + w_i \text{ if } d_i \geq d[i-1][k-1] \end{cases}$$

It takes $\Theta(n^2)$.

We have to find the optimal ordering (the greedy part). Let's take the boxes A and B :

- We can say that is more convenient to take one of the two based on the relation $d_A - w_B \leq d_B - w_A$.
- We can refactor the relation to $d_A + w_A \leq d_B + w_B$ so that it depends on only one element and so it can be used as a sorting criteria.

⁴Usually they are bad because of sparse memory allocation, so the CPU cannot fit multiple elements in a single cache line and, more importantly, cannot pre-fetch the next one.